Backtesting platform UI/UX design plan: addendum

Oct 7, 2026 · @qiwen

Purpose

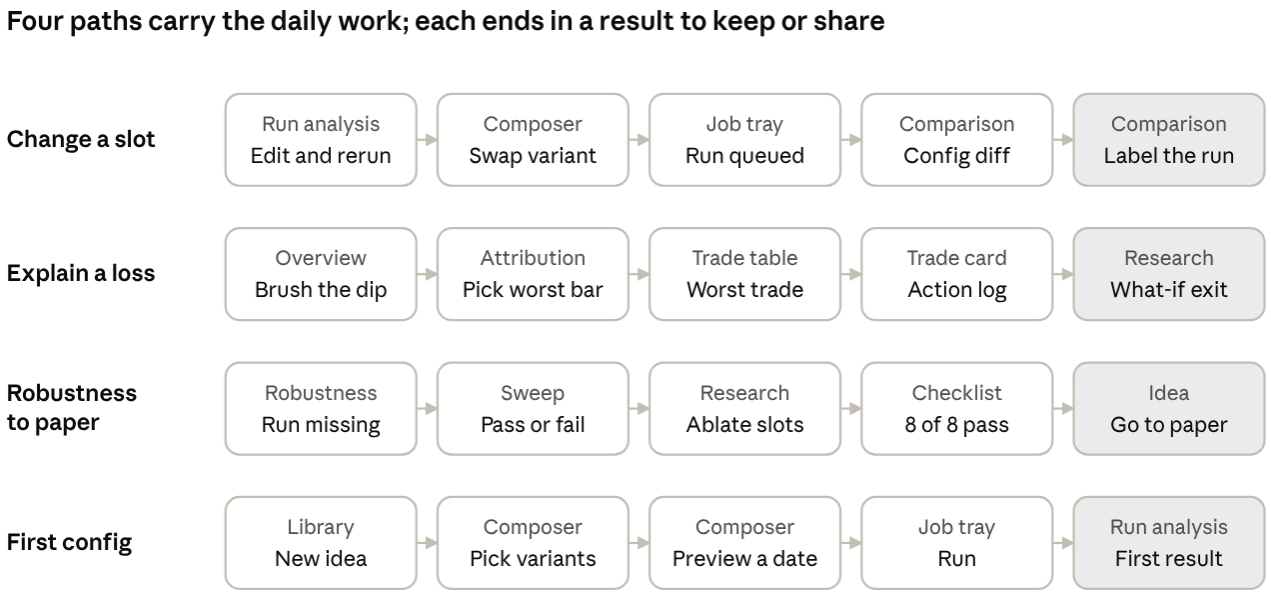
This addendum supplements *Backtesting platform UI/UX design plan* without changing its structure. It adds the four things the plan describes only in prose: how a researcher moves through the product step by step, what the key pages look like, how data flows inside the frontend, and how numbers and words are written on screen.

Part	Adds	Supplements plan section
A. Key flows	Four core paths drawn and broken into steps	3 Information architecture, 10 Interaction rules
B. Page wireframes	Composer, run analysis and trade card layouts	4, 6 and 7
C. Frontend architecture and data flow	Data layers, state ownership, loading and caching, performance budget, interaction feel, stack	11 State design, 14 Backend contract
D. Content and formatting rules	Numbers, units, dates, ids, labels, microcopy	12 Visual system

Everything here follows the plan's principles: runs are immutable, the backend computes every number, realism assumptions are tested and never tuned, and the research protocol is built into the interface.

A. Key flows

Four paths carry most of the daily work; each ends in something the researcher can keep or share, usually a comparison link. The diagram shows them at a glance and the tables give each step: where the user is, what they do, and what the screen does in return.



Key flows · 4 paths, 5 steps each

Each lane reads left to right; the tinted box is what the path leaves behind, usually a link someone else can open.

Flow 1: change one slot and compare

The most frequent path. Target: 2 steps to submit, 1 click from the finished run to its comparison.

#	Where	User does	Screen responds
1	Run analysis	Clicks "Edit and rerun" in the header, or a slot name anywhere on the page	Composer opens with this run's config as the baseline; the clicked slot's card is open
2	Composer	Swaps the variant or changes a parameter	A baseline dot appears on the field; the top line reads "Differs from baseline in 1 slot"
3	Composer (optional)	Runs a preview for a date	Candidate states and selected contracts for that date within 10 s
4	Composer	Presses Run (Cmd + Enter)	Job tray shows the run queued, then progress; the composer stays open for the next variation

#	Where	User does	Screen responds
5	Job tray	Clicks "Compare with baseline" on the finished toast	Comparison: config diff with the one slot highlighted, metrics with intervals, trade-level diff
6	Comparison	Labels the run "candidate" or "rejected", or opens its Robustness tab	Label shows in the log; the comparison URL is the shareable answer

Flow 2: explain a loss

From a dip on the equity curve to the component action behind it, without leaving the page.

#	Where	User does	Screen responds
1	Overview tab	Drags across the drawdown on the underwater chart	Every layer zooms to the range; the trade table filters; a range chip appears in the top tier
2	Attribution tab	Switches tab (range kept)	Greeks waterfall and P&L by closing component for the range
3	Attribution tab	Clicks the largest bar, e.g. a slot or vega	Trade table filters to trades where that slot or Greek drove the loss, worst first
4	Trade table	Presses Enter on the first row	Trade card opens; the chart shades its holding period
5	Trade card	Reads the lifecycle timeline and clicks the closing action	Action log highlights the row with reason and triggering data values; context values at that moment
6	Trade card (optional)	Runs a counterfactual exit	Paired run created; result labeled in-sample, opened in comparison

Flow 3: from a promising run to paper trading

The gate every strategy passes before it touches the paper account.

#	Where	User does	Screen responds
1	Robustness tab	Clicks "Run missing items"	Dialog lists the runs to be created and the trial count before and after; confirm submits a sweep
2	Sweep view	Waits or keeps working	Grid fills as members finish; each robustness item turns pass or fail
3	Robustness tab	Opens a failed item	The item's chart and a link to its comparison
4	Run analysis	Ablates or swaps weak slots; runs interaction tests	Paired runs in comparison; joint-effect heatmaps flag slots to tune together
5	Pre-launch checklist	Reaches 8 of 8 pass; reveals the hold-out once	Confirmation dialog; reveal logged with user and date; hold-out results fill in
6	Pre-launch checklist	Clicks "Promote to paper"	A paper-mode run appears in the same idea with the same analysis page

Flow 4: first config for a new idea

The path a new team member takes; target under 10 minutes without help.

#	Where	User does	Screen responds
1	Strategy library	Clicks "New idea", names it, sets the hold-out period	The idea is created with its hold-out locked
2	Composer	Picks a registered variant per slot	Schema forms with defaults; neutral variants pre-filled where nothing is picked
3	Composer	Reviews lifecycle, timing and data snapshot	Team live settings and the latest validated snapshot pre-filled; warnings if they differ
4	Composer	Previews a date	Candidates, selected contracts and an action log explain what the config does

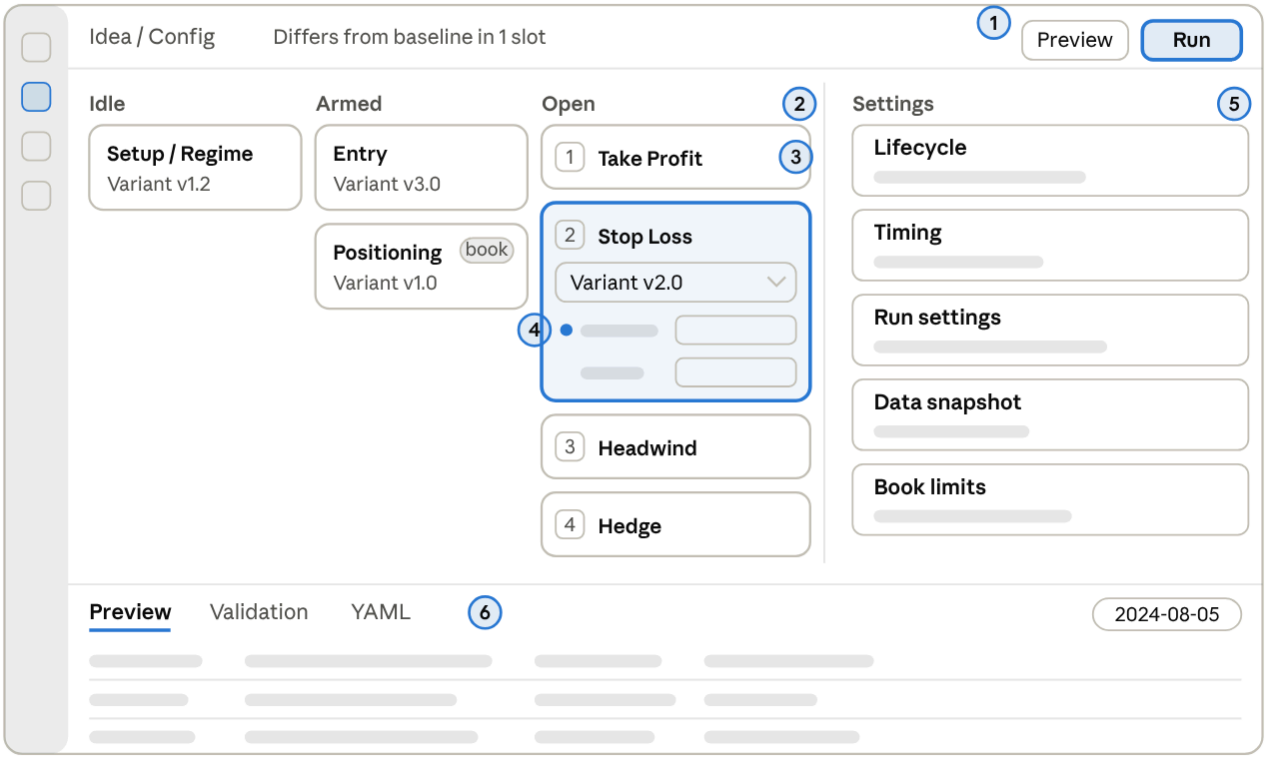
#	Where	User does	Screen responds
5	Composer	Runs	Job tray, then run analysis; empty states point to "Run the robustness set"

B. Page wireframes

Three wireframes fix the layout of the pages that carry most of the work. They show placement and hierarchy at desktop width (1440 px), not final styling; numbered callouts match the notes under each.

Composer

Composer: the slot board follows a trade from Idle to Open



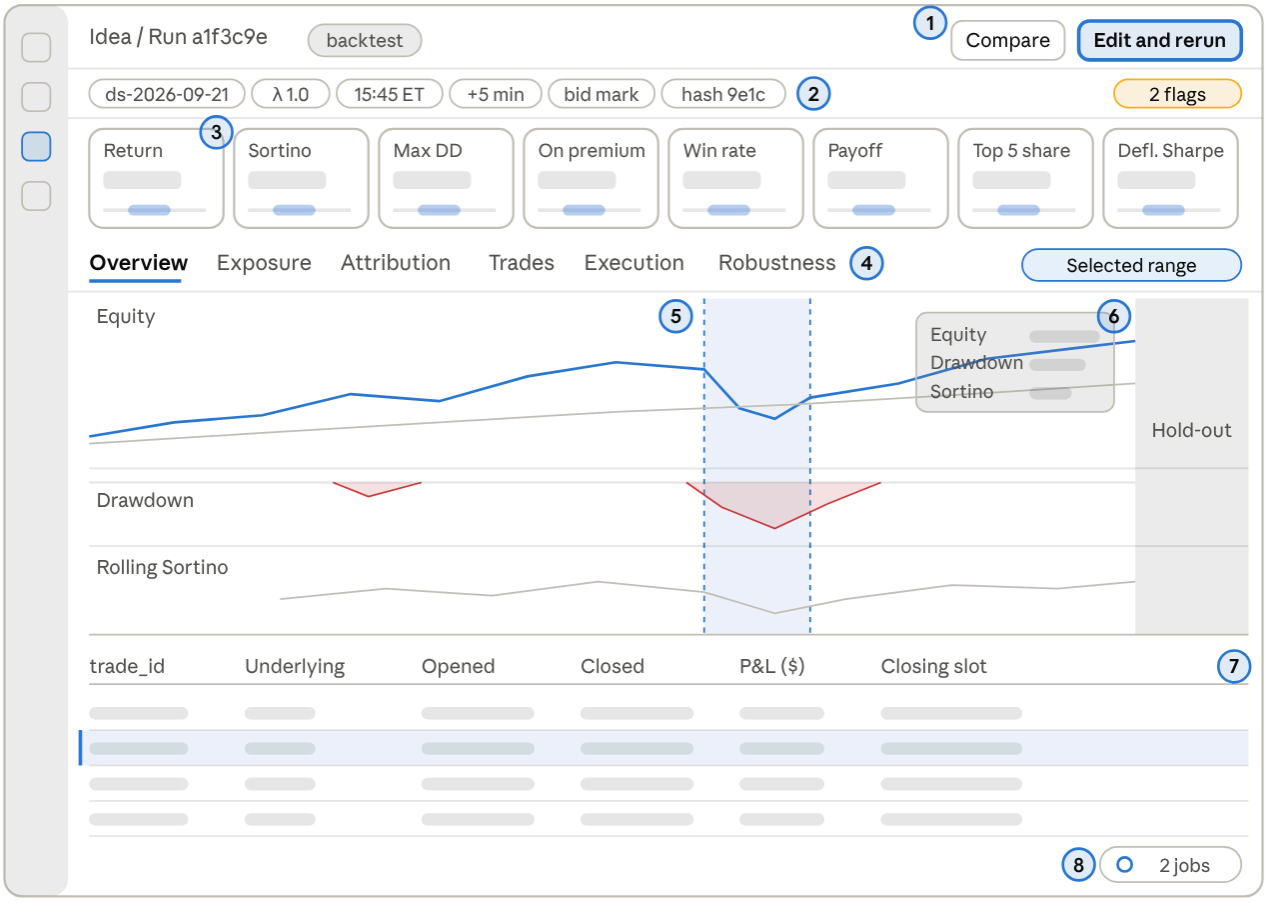
Composer wireframe · Stop Loss card open

1. Context bar: idea and config name, "Differs from baseline in 1 slot", Preview and Run; Run is the only primary button.
2. Slot board in three lifecycle columns, Idle, Armed and Open, so the card order reads like a trade's life. The open card shows its variant picker and schema form in place.
3. Open-state cards show their `open_state_order` as numbers; dragging reorders them.
4. A changed field carries a baseline dot; hover shows the baseline value and reset.

5. Settings panel: lifecycle, timing, run settings, data snapshot, book limits, collapsed to one-line summaries.
6. Bottom panel: Preview, Validation and YAML tabs; preview lists candidates and selected contracts for the chosen date.

Run analysis

Run analysis: conclusion on top, one time axis, trades at the foot



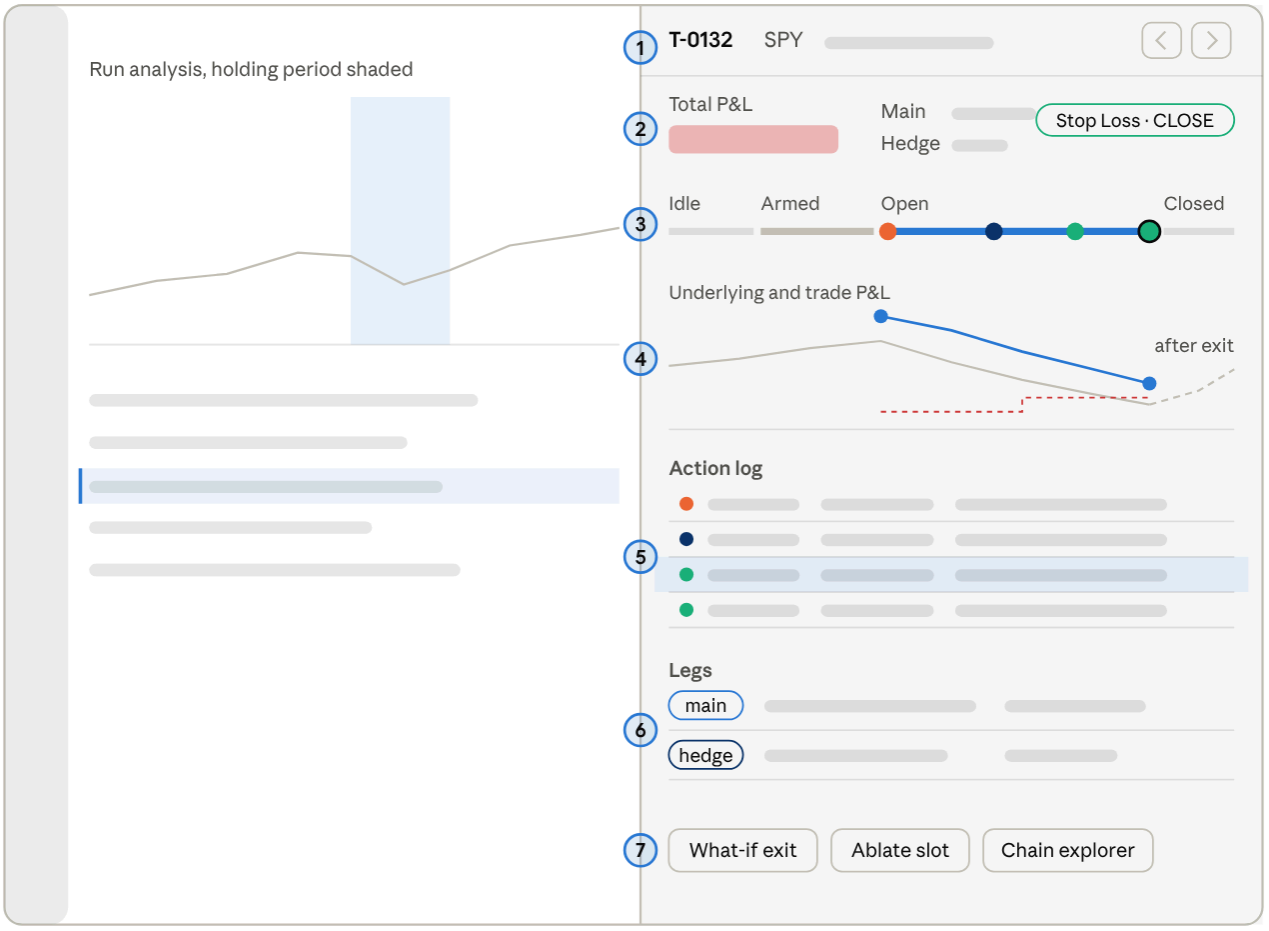
Run analysis wireframe · Overview tab, a range brushed

1. Context bar: idea and run breadcrumb, mode badge, Compare, Edit and rerun.
2. Realism strip, read-only, ending in the result hash; protocol flags beside it.
3. Metric cards, eight at most, each with its interval bar and baseline delta; the hero metric sits first.
4. Tabs; switching keeps range, selection and collapsed layers.
5. Chart stack on one time axis: equity with the hold-out hatched, drawdown, then the tab's own layers; the brushed range is shaded across all of them.
6. Fixed tooltip, top right of the stack, listing every layer's value at the hovered date.
7. Trade table filtered by the brush; the selected row shades its holding period above.

8. Job tray, collapsed to a count when idle.

Trade card

Trade card: from the result to the action that caused it, in place



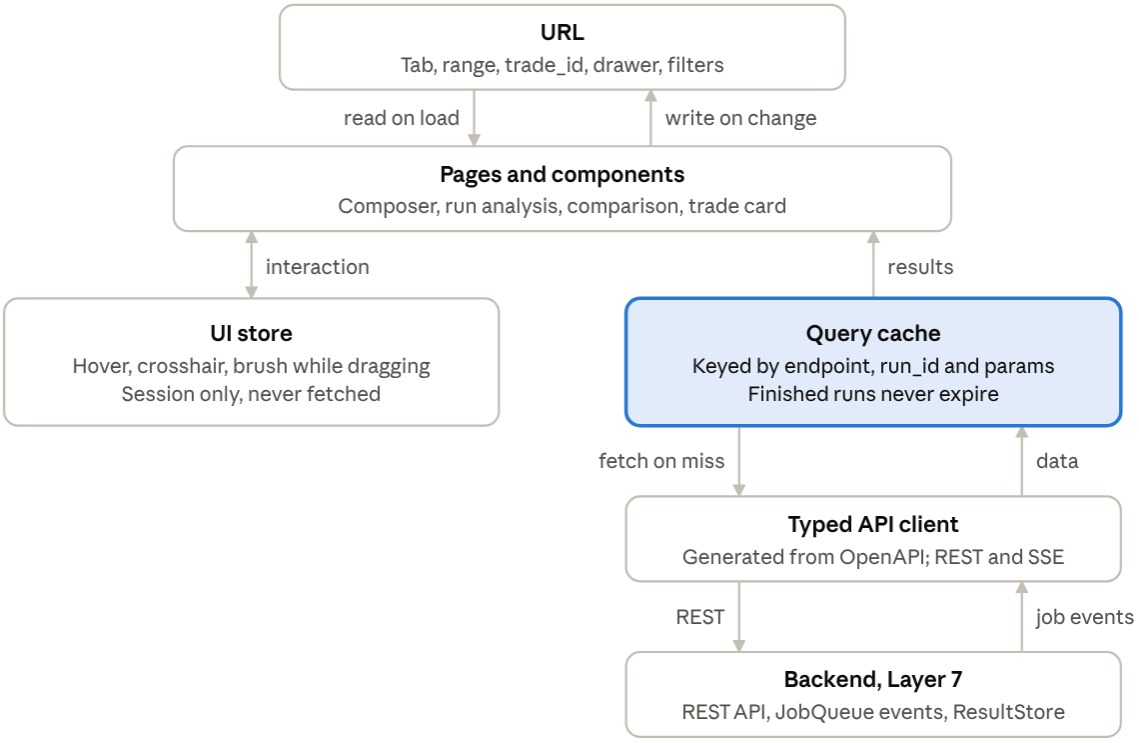
Trade card wireframe · drawer open over run analysis

1. Sticky header: `trade_id`, underlying, dates, previous and next (J and K).
2. Conclusion: total P&L, split into main and hedge legs, with the closing slot and action.
3. Lifecycle timeline: Idle, Armed, Open, Closed, with one marker per action in its slot's color.
4. Price and P&L over the hold; stops drawn as steps; shadow tracking after exit dashed.
5. Action log; the row for the clicked marker is highlighted with its reason and triggering values.
6. Legs with role, quotes, fill and spread paid.
7. Actions: counterfactual exit, ablate this slot, open in chain explorer.

C. Frontend architecture and data flow

A finished run never changes, so its data is cached by `run_id` forever; view state lives in the URL; only fleeting interaction state lives in a client store; and no layer derives a number the backend did not return.

View state lives in the URL, results in a cache keyed by run_id



Frontend data flow · URL, two stores, one typed client

Pages read view state from the URL and results from the cache; a job event from the backend invalidates only that run's entries.

State ownership

Every piece of state has exactly one owner, which removes most sync bugs.

State	Owner	Examples	Lifetime
View state	URL	Tab, time range, selected `trade_id`, drawer, filters, compared runs, baseline column	Shareable; back and forward work

State	Owner	Examples	Lifetime
Run data	Query cache, keyed by endpoint, `run_id` and params	Equity, snapshots, tradebook, metrics, attribution, events	Finished runs never expire; running jobs refetch on events
Interaction state	UI store	Hovered date, crosshair, brush while dragging, open menus	Current session
Preferences	User settings on the server	Metric card set, density, panel sizes, P&L color convention	Per user, every device
Drafts	Draft config on the server	Composer edits not yet submitted	Auto-saved per user
Job status	Event stream into the query cache	Queued, progress, finished, failed	Live

Loading and caching rules

1. Load in reading order: run summary and metrics, then equity and drawdown, then the active tab's layers, then the trade table. Other tabs prefetch when the user hovers them.
2. One fetch per series per page. Every chart layer reads the same cached series; brushing filters in memory and never refetches.
3. Zoom loads detail: the full period at overview resolution first, the brushed range at full resolution when zoomed. Downsampling is a drawing step only; tooltips and tables always show the exact stored values.
4. Trade tables and action logs are virtualized; sorting and filtering move to the server above 5,000 rows.
5. Job progress arrives by server-sent events (polling as fallback). A "finished" event invalidates that run's queries and raises the toast.
6. Optimistic updates only for metadata: labels, notes, archive. Results are never shown before the backend returns them.
7. Failures stay local: a layer that fails shows its own error and retry while the rest of the page renders.

Performance budget

Interaction	Budget
Crosshair and hover across the whole chart stack	One frame, under 16 ms
Brush release to every layer and the table updated	< 100 ms
Tab switch with cached data	< 100 ms
Open a trade card	< 150 ms cached, < 500 ms uncached
Revisit a finished run	First paint < 1 s
Analysis interactive once results are ready	< 2 s for 5 years and 1,000 trades
Composer field validation	< 50 ms
Preview	⩽ 10 s
Initial JavaScript	< 300 KB gzipped; chart libraries loaded per tab

Interaction feel

- Every click is acknowledged within 100 ms: a pressed state, a skeleton or a queued row in the job tray.
- Motion only for changes of place: drawer, panel, tab underline, 150 to 200 ms. Data never animates.
- No layout shift: skeletons reserve the final size and numbers use tabular figures, so values do not jitter as they load.
- Closing the drawer returns focus and scroll to where the user was.
- Long work is a job, never a frozen page: preview, runs and sweeps all run in the background.
- Reversible metadata changes (archive, label) offer Undo in a toast instead of a confirm dialog; confirm dialogs are kept for the irreversible, such as revealing the hold-out.

Recommended stack

Concern	Recommendation	Why
Framework	React with TypeScript	Typed end to end against the API contract

Concern	Recommendation	Why
Routing and URL state	TanStack Router with typed search params	The URL is typed state, not strings
Server data	TanStack Query	Keyed caching, invalidation, prefetch on hover
UI state	Zustand with selectors	Hover and crosshair updates do not re-render the page
Time-series stack	uPlot	Built for large time series with synced cursors across charts
Other charts	visx or ECharts for heatmaps, waterfalls, Gantt and surfaces	Custom marks in slot and Greek colors
Tables	TanStack Table with TanStack Virtual	Headless and virtualized
Forms from `ParamSchema`	JSON Schema exported from the pydantic models, rendered by one form component	One source of truth for validation and forms
YAML view	Monaco with a YAML language server fed the config JSON Schema	Errors pinned to lines as the user types
Base components	Radix primitives (or shadcn/ui) styled with the plan's tokens	Accessible behavior without a visual opinion
API types	OpenAPI spec generated from the backend, typed client via openapi-typescript	Contract changes break the build, not the page
Testing	Storybook, Playwright with screenshot tests, axe-core	States, visual regressions and accessibility in CI

D. Content and formatting rules

One format per kind of value, one word per concept, everywhere: researchers compare numbers across pages and runs, and a value written two ways reads as two values.

Numbers

Value	Rule	Example
Money (P&L, premium, fees)	Dollar sign, thousands separators, sign always shown; whole dollars at $1,000 and above, cents below	+$12,480 · −$3.15
Option prices and fills	Two decimals, as quoted	4.35
Returns and rates	Percent with one decimal; two below 1%	18.4% · 0.35%
Ratios (Sortino, Sharpe, Deflated Sharpe, payoff)	Two decimals	1.32
Intervals	Value, then the range in brackets	1.32 [0.84, 1.77]
Contract Greeks	Delta and gamma two decimals; vega and theta in dollars per contract	Δ 0.42 · Θ −$0.08 / day
Book Greeks	Dollar terms with the unit named	Vega $1,240 per vol point · Theta −$310 per day
Implied volatility	Percent with one decimal; IV rank as a 0 to 100 percentile without %	IV 24.6% · IV rank 72
Costs and slippage	Basis points	3.5 bp
Counts	Thousands separators	1,046 trades
Missing values	Empty cell, reason on hover	—

Negatives use the true minus sign (−) plus color; units go in column headers ("P&L ($)"), not in every cell; numbers are right-aligned in tabular figures.

Dates and time

- Market times are US Eastern and say so, on a 24-hour clock: 15:45 ET.
- Tables, ids and URLs use ISO dates (2026-10-07); prose and tooltips use Oct 7, 2026; chart axes use Oct 7.
- Expiries show the date and days to expiry together: 2026-11-20 · 44 DTE.

- Day counts state their convention in the header, trading or calendar, matching the backend's single time convention.
- Relative times ("3 min ago") only for job events, with the absolute time on hover.

Identifiers

Object	Shown as	Example
Run	First 7 characters of `run_id`, monospace; full id on hover and copy	a1f3c9e
Trade	`trade_id` as returned, monospace	T-0132
Contract	Readable form, OCC symbol on hover and copy	SPY 2026-11-20 580 C
Variant	Name and version	trend_breakout v1.3
Data snapshot	Snapshot ID, monospace	ds-2026-09-21

Terms

The interface uses the architecture's vocabulary, so screens, logs, configs and the API say the same thing.

Use	Not	Note
Slot	Module, component	The position in the strategy (Setup / Regime, Entry, …)
Variant	Plugin, implementation	What fills a slot, with a version
Run	Backtest result	"Job" only for queue status
Idea	Project, experiment	The group that owns the trial count and hold-out
Trade	Position	A position is a holding in accounting; a trade is a `trade_id`
Closing slot and action	Exit reason	E.g. "Stop Loss · CLOSE"
Realism assumption	Parameter	λ, latency, participation cap, mark method

Use	Not	Note
Hold-out	Test set	Locked per idea, revealed once
Paper, live	Simulated, real	Run modes

Microcopy

- Buttons say verb and object: Run, Compare with baseline, Ablate Stop Loss, Reveal hold-out. Never OK or Submit.
- Errors say what happened and what to do next: "Preview failed: snapshot ds-2026-09-21 has no quotes on 2024-08-05. Pick another date or snapshot."
- Warnings name the number: "Only 38 trades; conclusions need at least 60." (the minimum is a team setting)
- Confirm dialogs state the consequence: "Reveal the hold-out for this idea? This can be done once and is logged."
- Empty states say what will appear and offer one action.
- Sentence case everywhere; no exclamation marks; no "please".